

Autonomous Perception Ensemble

The Dataset - BDD100K

Berkeley DeepDrive 100K, a large-scale autonomous driving dataset with approximately 70,000 training images and 10,000 validation images captured from dashcam footage. It covers 10 object classes: car, truck, bus, person, rider, bicycle, motorcycle, traffic light, traffic sign, and train. You chose it because it's real-world driving data with diverse conditions different lighting, weather, occlusion not a clean lab dataset. It also provides annotations for multiple perception tasks (detection, segmentation, drivable area), which made it ideal for a multi-model ensemble.

The Three Models

Here's the breakdown of the neural network architecture behind each model in your ensemble.

RT-DETR-s (Object Detection) - Hybrid CNN + Transformer

This is a two-stage architecture. The backbone is a **ResNet** (CNN) that extracts multi-scale visual features from the input image through convolutional layers with skip/residual connections. Those feature maps then feed into a **Transformer encoder-decoder**. The encoder uses self-attention to reason about global context across the entire image this is why it handles occlusions better than pure CNN detectors, because attention lets distant regions of the image inform each other. The decoder uses learned object queries (similar to DETR) that each "ask" the encoded features whether an object exists at a particular location. The output is a set of detections directly no NMS needed because each query is responsible for at most one object. So in summary: CNN for local feature extraction, Transformer for global reasoning, end-to-end prediction.

U-Net (Drivable Area Segmentation) — Fully Convolutional Encoder-Decoder CNN

U-Net is a symmetric **encoder-decoder CNN** with skip connections between corresponding encoder and decoder layers. The encoder is a contracting path successive convolution and max-pooling layers that downsample the image and capture increasingly abstract features (edges → textures → shapes → semantic concepts). The decoder is an expanding path — upsampling layers (transposed convolutions) that gradually recover spatial resolution. The critical innovation is the skip connections at each level, the encoder's feature maps are concatenated directly to the decoder's feature maps at the same resolution. This lets the decoder use both high-level semantic information from the deep layers and fine-grained spatial information from the shallow layers, which is essential for producing precise pixel-level segmentation boundaries. Your U-Net was trained on BDD100K's drivable area masks to classify each pixel as drivable vs background.

MiDaS (Monocular Depth Estimation) - Transformer-based (DPT)

MiDaS uses a **Dense Prediction Transformer (DPT)** architecture. The backbone is a **Vision Transformer (ViT)** - the image is split into patches (typically 16×16), each patch is linearly embedded and fed through transformer encoder layers with self-attention, producing rich feature representations at multiple scales. These multi-scale features are then passed through a convolutional decoder head (similar in spirit to U-Net's decoder) that reassembles them into a full-resolution depth map. The key strength of using a transformer backbone for depth estimation is that self-attention captures long-range dependencies understanding that a distant building is far away requires relating it to nearby objects for scale context, which local convolutions struggle with. MiDaS was also trained on a mix of multiple depth datasets, making it robust to diverse scenes without fine-tuning.

The Challenges

The data preprocessing problem was a significant debugging effort. When you first set up the BDD100K dataset on Colab, only 1,156 training images were being found instead of 70,000. The root cause turned out to be that BDD100K's zip file stores train images across four subdirectories (`trainA/`, `trainB/`, `testA/`, `testB/`) inside `images/100k/train/`, and a non-recursive `*.jpg` glob only caught the files sitting directly in the `train/` directory. The fix required recursive glob patterns and flattening all subdirectory images into the top-level directories so that Ultralytics' `images/ ↔ labels/` path-swapping convention would work correctly with the flat label `.txt` files.

The Colab training interruption was another real challenge. Your RT-DETR training run on Colab got disconnected, so you had to work with the best model weights saved up to epoch 16 rather than completing the full 30-epoch training run. This is a practical reality of training on Colab — and it's actually a good interview talking point because it shows you understand checkpointing, resumability, and working with the best available model rather than insisting on ideal conditions.

The SAM segmentation dataset issue was a separate debacle. You initially tried to use a different dataset for the segmentation component. I believe you explored an alternative that turned out to be trained on streets of London but it created a domain mismatch with the BDD100K driving images which are primarily US roads. The visual characteristics, road markings, driving side, and scene composition were different enough that it wasn't working well, so you switched back to using BDD100K's own drivable area masks for consistency across the ensemble. This is actually a great example of understanding domain shift in practice.

The Integration Architecture

The ensemble is built as a `PerceptionEnsemble` class in `src/inference.py`. All three models are exported to ONNX format and loaded via ONNX Runtime, which means inference doesn't depend on PyTorch or TensorFlow at runtime just `onnxruntime`, making it lightweight and deployment-friendly.

The pipeline flow for a single image works like this:

An input dashcam image comes in. First, it goes through **RT-DETR** (loaded as the ONNX-exported detection model) which produces bounding boxes with class IDs, confidence scores, and coordinates (x1, y1, x2, y2) for each detected object across the 10 BDD100K classes. Second, the same image goes through **U-Net** (you used U-Net for drivable area segmentation rather than SAM for this particular pipeline) which outputs a binary segmentation mask each pixel labeled as drivable (1) or background (0), using BDD100K's drivable area annotations (red = directly drivable, blue = alternatively drivable). Third, the image goes through **MiDaS** which outputs a relative depth map a per-pixel estimate of how far each part of the scene is from the camera.

These three outputs are stored in a **PerceptionResult** dataclass that holds the list of **Detection** objects, the segmentation mask as an H×W numpy array, the depth map as an H×W numpy array, and optionally a fused visualization image. Each **Detection** also has an optional **depth** field by sampling the depth map at the center of each bounding box, you can estimate how far each detected object is.

The fusion step composites everything into a single visualization: the original image overlaid with green for drivable area, bounding boxes with class labels and confidence scores drawn on top, and a depth colormap blended in. This gives you a single frame that shows what the system sees, what it can drive on, and how far things are.

The Gradio Demo

The Gradio app lives in [demo/app.py](#). It loads the **PerceptionEnsemble** once globally at startup (so models aren't reloaded per request), then exposes a simple interface: upload a dashcam image, and get back the fused visualization. The demo includes toggle checkboxes — show detections, show segmentation, show depth so you can view each perception layer independently or combined. The segmentation overlay is green with 30% opacity blended on the original, the depth overlay uses a plasma colormap at 40% opacity, and detection boxes are drawn with class-colored bounding boxes and confidence labels.

Demographic-Aware Face Generation

The Project

A conditional GAN trained on the FairFace dataset (55,000+ images across 7 demographic groups) to generate realistic face images conditioned on ethnicity. The goal was demographically-aware generation — ensuring the model could produce quality outputs across all groups rather than just the majority class.

Architecture decisions

You used a Projection Discriminator rather than simple label concatenation for conditioning — this is a more principled approach where the class embedding is projected into the discriminator's feature space via an inner product, giving the discriminator a stronger conditioning signal. Spectral Normalization was applied to stabilize training by constraining the Lipschitz constant of the discriminator's weight matrices. You used hinge loss rather than the standard binary cross-entropy GAN loss, which tends to produce more stable training dynamics.

The core challenge - discriminator dominance

The discriminator was winning too easily, which meant the generator couldn't learn. The generator loss was climbing while the discriminator loss collapsed toward zero. You diagnosed this as a classic GAN training imbalance and fixed it with asymmetric learning rates — giving the generator a higher learning rate than the discriminator so it could "keep up." This reduced generator loss by 62%.

Training and evaluation

You trained on Google Colab using NVIDIA A100 GPUs (and H100 on your resume). You instrumented the training loop to checkpoint every epoch and evaluate FID scores periodically, which let you identify epoch 33 as the best-performing checkpoint rather than guessing. You also built a per-ethnicity FID evaluation framework — rather than just computing a single FID score across all generated images, you computed FID separately for each demographic group to measure generation fairness. This is the "fairness-aware evaluation" angle that makes the project more than just another GAN project.

The mode collapse issue

Earlier in the project you encountered mode collapse, the generator producing similar-looking outputs regardless of the conditioning label. Adding Spectral Normalization and the Projection Discriminator helped resolve this, along with the asymmetric learning rates.

Your retrospective takeaway

You mentioned that if you were to do it again, you would establish quantitative evaluation baselines (FID and IS scores) from the very first training run rather than adding them

mid-project, since doing so would have surfaced the mode collapse issue earlier and reduced the number of diagnostic iterations needed.

Prodigy Infotech (Jul - Sep 2023) - Machine Learning Intern

This was your earliest ML experience. The core project was a housing price prediction task — classical ML, not deep learning. You worked on a dataset with 10+ features (demographic and economic variables) and the interesting part was the diagnostic work upstream of modeling. You ran VIF (Variance Inflation Factor) analysis and correlation profiling to identify multicollinearity — meaning several features were highly correlated with each other, which inflates coefficient variance in linear models and makes them unreliable. Based on that diagnosis, you benchmarked OLS, Ridge, and Lasso regression using cross-validated RMSE. Ridge won because it handles multicollinearity through L2 regularization (shrinks correlated coefficients proportionally rather than eliminating them like Lasso's L1), giving you a 12% error reduction over the baseline. This was a 4-person team project.

The interview-relevant takeaway: you demonstrated that the modeling choice should be driven by the data characteristics you discover in EDA, not just by trying every algorithm blindly.

YBI Foundation (Nov 2023 - Aug 2024) - AI and Machine Learning Intern

Longer and more substantial. The core project was a multi-class text classification pipeline on 10,000+ customer feedback samples across 8 categories. You used TF-IDF for feature extraction (converting raw text into numerical vectors based on term frequency weighted by inverse document frequency). The key challenge was class imbalance — some feedback categories had far more samples than others, which meant the model was biased toward majority classes and underperforming on minority ones. You applied SMOTE (Synthetic Minority Oversampling Technique) to generate synthetic samples for underrepresented categories, which improved minority-class recall by 18%. Importantly, you didn't just try SMOTE blindly — you ran systematic ablation across different balancing strategies to determine that SMOTE was the best approach. You also built per-category precision-recall benchmarks rather than relying on a single aggregate accuracy number, and documented failure analysis to inform relabeling of ambiguous samples.

The interview-relevant takeaway: you built a complete pipeline end-to-end (not just a model), handled a real-world data quality issue (class imbalance) with a principled approach, and evaluated rigorously per-class rather than with a single metric.